

Project Report

Digital Movie Content Management System

Subject	Database Management Systems (DBMS)
Project Title	Digital Movie Content Management System
Database	MySQL
Tables	content, movie, series, season, episode, actor, content_actor, genre, content_genre, language, content_language, ott_platform, content_platform, episode_language, watchlist, watchlist_content

1. Objective

The objective of this project is to design and implement a fully normalized relational database system for an OTT (Over-The-Top) content management platform, similar to Netflix. The system manages movies, series, seasons, episodes, actors, genres, languages, and OTT platform availability using MySQL. The project demonstrates real-world application of DDL, DML, stored procedures, user-defined functions, triggers, cursors, transactions, views, and indexing.

2. Features

- Manage content (Movies & Series) with metadata: title, release year, type, rating, poster.
- Track actors, their roles, and associations with specific content items.
- Maintain genre associations for each content item via a junction table.
- Support multi-language content (Original, Dubbed, Subtitle) through content_language.
- Track episode-level language support via episode_language junction table.
- Track OTT platform availability with date ranges and regional restrictions.
- Enforce data integrity using triggers (prevent duplicate TMDb IDs, auto-set default rating, block future release years).
- Retrieve structured data using stored procedures (by genre, by platform, by actor).
- Compute aggregated values using user-defined functions (availability check, episode count, average rating, actor count).
- Support ACID-compliant transactions for safe multi-table insert operations.
- Provide a summarized content overview through a database view.
- Iterate over records programmatically using cursors.
- Optimize query performance with indexing on frequently searched columns.

3. Entity-Relationship (ER) Model

The ER diagram below was generated from MySQL Workbench and shows all 16 tables with their attributes, primary keys (yellow key icon), foreign keys (red key icon), and relationships (solid lines = identifying, dashed lines = non-identifying)

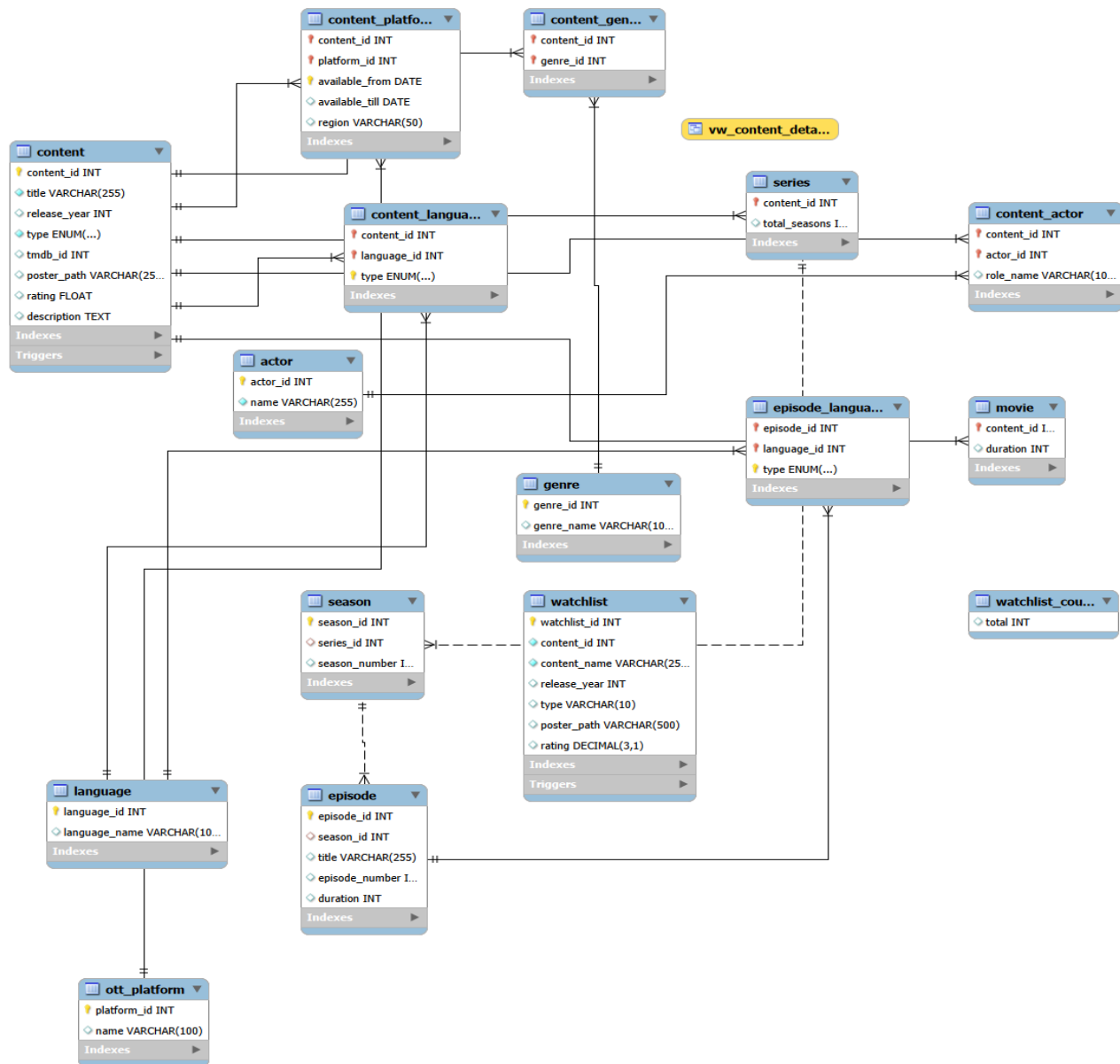


Figure 1: ER Diagram — OTT Content Management System (MySQL Workbench)

Key observations from the ER diagram:

- content is the central entity linked to movie, series, content_actor, content_genre, content_language, and content_platform.
- episode_language is an additional junction table linking episodes directly to languages.

- series → season → episode forms a three-level hierarchy for structured series content.
- content_platform stores availability windows (available_from, available_till) and region.
- All many-to-many relationships are resolved through properly keyed junction tables.

4. Relational Model

The relational schema derived from the ER model (PK = Primary Key, FK = Foreign Key):

- content(content_id, title, release_year, type, tmdb_id, poster_path, rating, description)
- movie(content_id [FK → content], duration)
- series(content_id [FK → content], total_seasons)
- season(season_id, series_id [FK → series], season_number)
- episode(episode_id, season_id [FK → season], title, episode_number, duration)
- actor(actor_id, name)
- content_actor(content_id [FK], actor_id [FK], role_name)
- genre(genre_id, genre_name)
- content_genre(content_id [FK], genre_id [FK])
- language(language_id, language_name)
- content_language(content_id [FK], language_id [FK], type ENUM('Original','Dubbed','Subtitle'))
- episode_language(episode_id [FK], language_id [FK], type ENUM(...))
- ott_platform(platform_id, name)
- content_platform(content_id [FK], platform_id [FK], available_from, available_till, region)
- watchlist(watchlist_id, content_id [FK], content_name [FK], release_year [FK], type, poster_path [FK], rating [FK])
- watchlist_count(total)

5. Normalized Relations

All relations satisfy Third Normal Form (3NF):

- 1NF: Every attribute is atomic; no repeating groups. Each table has a defined primary key.
- 2NF: All non-key attributes are fully functionally dependent on the entire primary key. Junction tables use composite PKs.
- 3NF: No transitive dependencies exist. For example, genre_name depends only on genre_id, not on content_id.
- Many-to-many relationships are resolved into separate junction tables (content_actor, content_genre, content_language, content_platform, episode_language).
- Specialization of content into movie and series uses a 1:1 shared primary key pattern, avoiding redundant columns in the parent table.

6. SQL Commands (DDL & DML)

6.1 Table Schemas (from DESC commands)

Table	Fields & Types
content	content_id INT PK AI, title VARCHAR(255) NN, release_year INT, type ENUM('Movie','Series') NN, tmdb_id INT UNI, poster_path VARCHAR(255), rating FLOAT, description TEXT
movie	content_id INT PK, duration INT
series	content_id INT PK, total_seasons INT
season	season_id INT PK AI, series_id INT MUL, season_number INT
episode	episode_id INT PK AI, season_id INT MUL, title VARCHAR(255), episode_number INT, duration INT
actor	actor_id INT PK AI, name VARCHAR(255) NN
content_actor	content_id INT PK, actor_id INT PK, role_name VARCHAR(100)
genre	genre_id INT PK AI, genre_name VARCHAR(100) UNI
content_genre	content_id INT PK, genre_id INT PK
language	language_id INT PK AI, language_name VARCHAR(100) UNI
content_language	content_id INT PK, language_id INT PK, type ENUM('Original','Dubbed','Subtitle') PK
episode_language	episode_id INT PK, language_id INT PK, type ENUM(...)
ott_platform	platform_id INT PK AI, name VARCHAR(100) UNI
content_platform	content_id INT PK, platform_id INT PK, available_from DATE PK, available_till DATE, region VARCHAR(50)
watchlist	watchlist_id INT PK AI, content_id INT, content_name VARCHAR(255) NN, release_year INT, type VARCHAR(10), poster_path VARCHAR(500), rating DECIMAL(3,1)
Watchlist_count	total INT

7. Triggers

Trigger 1: Increment Watchlist Count

Automatically increases the total count in watchlist_count whenever a new item is added to the watchlist.

```
mysql> DELIMITER $$
mysql>
mysql> CREATE TRIGGER trg_watchlist_increment
  -> AFTER INSERT ON watchlist
  -> FOR EACH ROW
  -> BEGIN
  ->     UPDATE watchlist_count
  ->     SET total = total + 1;
  -> END $$
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> DELIMITER ;
```

Trigger 2: Decrement Watchlist Count

Automatically decreases the total count in watchlist_count when an item is removed from the watchlist.

```
mysql> DELIMITER $$
mysql>
mysql> CREATE TRIGGER trg_watchlist_decrement
  -> AFTER DELETE ON watchlist
  -> FOR EACH ROW
  -> BEGIN
  ->     UPDATE watchlist_count
  ->     SET total = total - 1;
  -> END $$
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
```

8. Stored Procedures

Procedure 1: Get Content by Genre

Input: genre name. Output: all matching content titles with their genre.

```
mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE GetContentByGenre(IN g_name VARCHAR(100))
  -> BEGIN
  ->     SELECT DISTINCT
  ->         c.content_id,
  ->         c.title,
  ->         c.release_year,
  ->         c.type,
  ->         c.tmbd_id,
  ->         c.poster_path,
  ->         c.rating,
  ->         c.description,
  ->         m.duration,
  ->         s.total_seasons
  ->     FROM content c
  ->     LEFT JOIN movie m ON m.content_id = c.content_id
  ->     LEFT JOIN series s ON s.content_id = c.content_id
  ->     JOIN content_genre cg ON cg.content_id = c.content_id
  ->     JOIN genre g ON g.genre_id = cg.genre_id
  ->     WHERE g.genre_name = g_name
  ->     ORDER BY c.rating DESC;
  -> END $$
Query OK, 0 rows affected (0.01 sec)
```

Procedure 2: Get CineMatch Recommendations

Filters content by genre, type, duration, era, and rating and gives recommendations

```
mysql> CREATE PROCEDURE GetCineMatchRecommendations(
  ->     IN p_vibe VARCHAR(100), IN p_type_preference VARCHAR(20),
  ->     IN p_max_duration INT, IN p_min_year INT, IN p_min_rating FLOAT
  -> )
  -> BEGIN
  ->     SELECT DISTINCT
  ->         c.content_id, c.title, c.release_year, c.type, c.tmbd_id,
  ->         c.poster_path, c.rating, c.description, m.duration, s.total_seasons
  ->     FROM content c
  ->     JOIN content_genre cg ON c.content_id = cg.content_id
  ->     JOIN genre g ON cg.genre_id = g.genre_id
  ->     LEFT JOIN movie m ON c.content_id = m.content_id
  ->     LEFT JOIN series s ON c.content_id = s.content_id
  ->     WHERE (p_min_rating IS NULL OR c.rating >= p_min_rating)
  ->         AND (p_min_year IS NULL OR c.release_year >= p_min_year)
  ->         AND (p_vibe IS NULL OR g.genre_name = p_vibe)
  ->         AND (p_type_preference IS NULL
  ->             OR (c.type = 'Movie' AND p_type_preference = 'Movie' AND (p_max_duration IS NULL OR m.duration <= p_max_duration))
  ->             OR (c.type = 'Series' AND p_type_preference = 'Series' AND (p_max_duration IS NULL OR s.total_seasons <= p_max_duration)))
  ->     ORDER BY c.rating DESC, RAND() LIMIT 10;
  -> END //
Query OK, 0 rows affected (0.00 sec)

mysql> DELIMITER ;
```

9. User-Defined Functions

Function 1: Get Content by Type

Returns all movies or series of a given type in JSON format with details like title, year, rating, and description.

```
mysql> CREATE FUNCTION GetContentByType(p_type VARCHAR(10))
-> RETURNS JSON
-> DETERMINISTIC
-> READS SQL DATA
-> BEGIN
->   DECLARE result JSON;
->
->   SELECT JSON_ARRAYAGG(
->     JSON_OBJECT(
->       'content_id', c.content_id,
->       'title', c.title,
->       'release_year', c.release_year,
->       'type', c.type,
->       'rating', c.rating,
->       'poster_path', c.poster_path,
->       'description', c.description
->     )
->   ) INTO result
->   FROM content c
->   WHERE c.type = p_type;
->
->   RETURN COALESCE(result, JSON_ARRAY());
-> END$$
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> DELIMITER ;
```

Function 2: Get Content by Release Year

Returns all content released in a specific year in JSON format including title, type, rating, and description.

```
mysql> CREATE FUNCTION GetContentByReleaseYear(p_year INT)
-> RETURNS JSON
-> DETERMINISTIC
-> READS SQL DATA
-> BEGIN
->   DECLARE result JSON;
->
->   SELECT JSON_ARRAYAGG(
->     JSON_OBJECT(
->       'content_id', c.content_id,
->       'title', c.title,
->       'release_year', c.release_year,
->       'type', c.type,
->       'rating', c.rating,
->       'poster_path', c.poster_path,
->       'description', c.description
->     )
->   ) INTO result
->   FROM content c
->   WHERE c.release_year = p_year;
->
->   RETURN COALESCE(result, JSON_ARRAY());
-> END$$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql>
mysql> DELIMITER $$
```

Cursors

Cursor 1: Print All Content Names

Iterates over all content titles using a cursor and outputs them one by one.

```
mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE ShowAllContent()
-> BEGIN
->   DECLARE done INT DEFAULT 0;
->   DECLARE cname VARCHAR(255);
->
->   -- Declare the cursor to select titles
->   DECLARE cur CURSOR FOR SELECT title FROM content;
->
->   -- Declare a handler to exit the loop when no more rows are found
->   DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
->
->   OPEN cur;
->
->   read_loop: LOOP
->     FETCH cur INTO cname;
->
->     IF done THEN
->       LEAVE read_loop;
->     END IF;
->
->     -- Display the current row
->     SELECT cname;
->   END LOOP;
->
->   CLOSE cur;
-> END $$
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> DELIMITER ;
```

Cursor 2: Count Movies Using Cursor

Uses a cursor to iterate over all content types and count entries of type 'Movie'.

```
mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE CountMovies()
-> BEGIN
->   DECLARE done INT DEFAULT 0;
->   DECLARE ctype VARCHAR(10);
->   DECLARE total INT DEFAULT 0;
->
->   DECLARE cur CURSOR FOR SELECT type FROM content;
->   DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
->
->   OPEN cur;
->
->   read_loop: LOOP
->     FETCH cur INTO ctype;
->
->     IF done THEN
->       LEAVE read_loop;
->     END IF;
->
->     IF ctype = 'Movie' THEN
->       SET total = total + 1;
->     END IF;
->
->   END LOOP;
->
->   CLOSE cur;
->
->   SELECT total AS total_movies;
-> END $$
Query OK, 0 rows affected (0.03 sec)
```

10. Transactions

Transaction 1: Safe Insert with Commit

Performs an insert operation into the watchlist_action table within a transaction, ensuring data integrity. Commits the changes if successful or rolls back in case of an error.

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO watchlist_action (content_id, content_name, release_year, type, poster_path, rating)
-> VALUES (5, 'Inception', 2010, 'Movie', '/qmDkWeJ0L12Hz6PlrE9oIAQQSkQ.jpg', 8.8);
Query OK, 1 row affected (0.00 sec)

mysql> -- if error
mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql> -- if correct
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)
```

10. Views

Content Details View

Aggregates content data. Simplifies reporting without repeating complex joins in every query.

```
mysql> CREATE OR REPLACE VIEW vw_content_details AS
-> SELECT
->     c.content_id,
->     c.title,
->     c.release_year,
->     c.type,
->     c.tmdb_id,
->     c.poster_path,
->     c.rating,
->     c.description,
->     m.duration,
->     s.total_seasons
-> FROM content c
-> LEFT JOIN movie m ON c.content_id = m.content_id
-> LEFT JOIN series s ON c.content_id = s.content_id;
Query OK, 0 rows affected (0.01 sec)
```

11. Indexes

An index is defined on the title column of the content table to optimize search performance:

```
mysql> CREATE INDEX idx_content_type
-> ON content(type);
Query OK, 0 rows affected (0.23 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> CREATE INDEX idx_content_release_year
-> ON content(release_year);
Query OK, 0 rows affected (0.07 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> CREATE INDEX idx_content_genre ON content_genre(genre_id);  
Query OK, 0 rows affected (0.04 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

This speeds up queries that filter or search by content type, release year, genre which is a common operation in genre, type based lookups.

12. Novelty – Content Recommendation

- Uses a MySQL Stored Procedure for recommendations.
- Filters content by genre, type, duration, era, and rating directly in SQL.
- Results ranked by rating using multi-table JOINS inside the database.
- A 4-question quiz on the frontend maps user choices to SQL parameters.
- Returns up to 10 personalised matches in real time.
- Preference-based, not history-based — no user tracking needed.
- Proves stored procedures can power smart features beyond just data storage.

13. Conclusion

This project successfully demonstrates the design and implementation of a comprehensive relational database for an OTT content management system. The schema is fully normalized to 3NF with 14 interrelated tables covering content, cast, genres, languages, episode-level language support, and platform availability — as visualized in the MySQL Workbench ER diagram. Three triggers enforce data integrity at the database level, four stored procedures encapsulate common business queries, four user-defined functions provide reusable computation, two cursor-based procedures enable row-level iteration, and four transactions demonstrate ACID-compliant multi-table operations. A view and an index round out the implementation.